

我用 Claude Skills 自动化代码审查，一周省了10个小时

上周五晚上11点，我又在看同事提的 PR。第27个了。眼睛都快看花了。

然后我发现一个变量名写错了，usreName 应该是 userName。这种低级错误，我已经指出过不下20次。突然很烦，为什么我要花时间指出这些机械性的问题？我应该关注的是架构设计、业务逻辑这些真正需要人类智慧的地方啊。

直到我接触到 Claude Skills，一切都变了。

什么是 Claude Skills？说人话

简单说，就是你可以教 Claude 做特定的事情，然后它就会记住这套流程。

比如你可以教它："看到 Python 代码，就检查这10个常见问题"，或者"审查前端代码时，重点看这5个方面"。下次你再让它审查代码，它就会自动按你教的套路来。

不用每次都重复说明，不用担心它忘记团队规范。一次配置，终身受用。

Anthropic 在10月份推出这个功能，我第一时间就试了。老实说，前几次有点磕磕绊绊，但现在已经成为我日常开发的必备工具了。

为什么我觉得 Skills 比其他工具好用

我之前用过 ESLint、SonarQube 这些静态分析工具。它们确实能抓到很多问题，但有个致命缺点：**只能检查写死的规则**。

比如你发现团队总是在某个地方犯同样的错误，想加一条新规则？得去研究怎么写插件，写完还得全员升级配置。太麻烦了。

Claude Skills 不一样。它理解代码的上下文，能做更灵活的判断。而且最爽的是，**你用自然语言告诉它要检查什么就行了**，不用写什么复杂的配置。

5分钟快速上手

第一步：开启功能

Settings → Features → 找到 Agent Skills，打开。就这么简单。

团队版用户可能需要问管理员要权限，但一般都会给的。

第二步：创建你的第一个审查 Skill

我第一次创建 Skill 的时候，直接问 Claude："帮我创建一个代码审查的 Skill"。

它会问你几个问题：

- 主要写什么语言？
- 重点检查哪些方面？
- 有没有团队特定的规范？

根据你的回答，Claude 会生成一个文件夹，里面包含：



```
code-review-skill/  
|—— SKILL.md      # 主要说明文档  
|—— references/    # 参考资料  
|   |—— coding-standards.md  
|   |—— security-checklist.md
```

你需要做的就是把团队的具体规范填进去。

第三步：写审查规则（核心部分）

这是最重要的一步。你要告诉 Claude 具体怎么审查代码。

我分享一下我自己的写法，你可以参考：



markdown

Code Review Skill

你要检查这些东西

第一遍快速扫描

- 这个 PR 改了多少行？（超过400行就提醒拆分）
- 提交信息写清楚了吗？
- 改的是哪个模块？

第二遍认真看

1. 逻辑对不对

- 有没有明显的 bug
- 边界条件考虑了吗
- 异常处理够不够

2. 安全问题

- SQL 注入风险
- XSS 漏洞
- 敏感数据有没有加密

3. 代码质量

- 变量命名清不清楚（别学那个 `usreName`）
- 有没有重复代码
- 注释够不够

最后给报告

- 严重问题（必须改）
- 建议改进（最好改）
- 写得好的地方（要夸）

注意，用你自己的话写。别照搬模板，要根据团队实际情况来。

第四步：测试

在真实项目里试试：



bash

```
claude-code -p "用我的 code-review skill 看看这个 PR"
```

第一次肯定会有问题。我当时发现它对某些 Python 的写法理解有偏差，后来在 Skill 里加了具体说明就好了。

多测几次，根据结果调整规则。这个过程就像调参数，需要耐心。

真实效果：我的一周使用体验

上周我统计了一下使用前后的数据，变化挺明显的。

使用前：

- 每个 PR 审查时间：平均 40–50 分钟
- 每天花在代码审查上的时间：2–3 小时
- 经常在晚上或周末才能回复 PR
- 指出重复性问题（命名、格式等）占了大半时间

现在：

- AI 先过一遍，过滤掉 70% 的基础问题
- 我只需要花 15–20 分钟看关键逻辑
- 半夜提交的 PR，早上就能看到 AI 的初步反馈
- 每周省出来大概 10 个小时

这 10 个小时我拿来干嘛了？写了两个一直想做但没时间做的功能，还学了一个新框架。真的赚到了。

但也不是完美的

说实话，AI 也会犯错：

1. **有时候太严格**。比如我写个快速原型，它还要求我加完整的错误处理。这时候我会在提示词里说"这是 prototype，放松点"。
2. **不懂业务逻辑**。比如我们有个特殊的权限规则，AI 第一次看总说有问题。后来我把业务规则写进 Skill 的 references 文件夹就好了。
3. **偶尔误报**。它可能把一些团队约定俗成的写法标记为"不规范"。这个需要人工判断。

所以我的用法是：**AI 做初筛，人做决策**。就像有个实习生帮你先看一遍，但最终决定还是你做。

几个实用技巧

1. 针对不同语言创建不同 Skill

我现在有三个 Skill：

- python-review：检查 Python 特有的问题（比如 GIL、内存管理）
- react-review：专门看 React 组件（hooks 依赖、性能优化）
- api-review：审查 API 设计（RESTful 规范、错误码）

Claude 会自动根据代码类型选择合适的 Skill，不需要你手动指定。

2. 集成到 CI/CD

如果你想让 AI 自动审查每个 PR，可以接入 GitHub Actions：



python

简化的示例

```
import anthropic

client = anthropic.Anthropic(api_key="YOUR_KEY")
response = client.messages.create(
    model="claude-sonnet-4-20250514",
    skills=["code-review"],
    messages=[{"role": "user", "content": f"审查这个 PR: {pr_diff}"}]
)
```

我们团队现在是：PR 提交后，AI 先审查，没问题才通知人工审查者。节省了很多时间。

3. 团队共享 Skills

把你的 Skill 文件夹放到 Git 仓库里，团队成员可以直接 clone 使用。

我们建了个 team-skills 仓库，里面有各种 Skills。新人入职第一天就能用上团队的最佳实践。

你可能想问的几个问题

Q：这玩意儿会取代人工审查吗？

不会。它只是个助手。就像你有个初级开发者帮忙看代码，但架构决策、业务判断还是得靠人。

我的建议是：让 AI 过滤基础问题，人专注高级问题。分工明确，效率更高。

Q：万一 AI 理解错了怎么办？

会发生的。所以你需要：

- 1. 在 Skill 里把团队特有的规则写清楚
- 2. 定期根据误报调整规则
- 3. 最终决策权在人，不要盲目相信 AI

Q：小团队值得用吗？

值得。我们才 5 个人，每个人每周都能省出好几个小时。时间就是钱啊。

而且配置 Skill 也就花半天时间，之后就是一劳永逸的事情。

写在最后

说实话，刚开始用 Claude Skills 的时候，我也半信半疑。觉得这又是一个"看起来很美"的玩具。

但用了一个月后，我真的回不去了。不是因为它有多完美，而是因为它确实解决了我的痛点：**让我把时间花在真正重要的事情上。**

不用再纠结变量名错了、忘加分号这种小事。不用晚上 11 点还在看 PR。可以把精力放在思考架构、优化性能、带新人这些更有价值的工作上。

如果你也被代码审查搞得焦头烂额，不妨试试 Claude Skills。花半天时间配置，之后每周省 10 个小时。这笔账怎么算都划算。

最后，我把自己的 Skill 放在 GitHub 上了：github.com/yourname/code-review-skills（示例链接）。你可以参考改成适合自己团队的版本。

有问题随时留言，我看到会回。

对了，你们团队现在是怎么做代码审查的？有没有什么痛点？评论区聊聊？